Lexify — Feature & Bug Tasks (Round 5: AI Translator, Avatar Bug, Contribution Points)

Written for direct use in Antigravity, referencing the current codebase.

---

1 & 2. Connect to AI API — Make Dictionary Work Like a Translator (Lecturer's Revision)

Current state: AI is only partially wired in. `AIService.java` already has a working `askAI()` method that calls OpenRouter, and it's currently used for exactly one thing: `/ai` gives a short Indonesian explanation of a single English word. **The Dictionary search itself (`/cari`) never touches AI at all** — it only checks the local `Kamus` in-memory maps, and if a word isn't found, it just returns nothing.

What the lecturer's revision is asking for: turn the dictionary from "a static list lookup" into "a real translator" — meaning it should be able to handle **anything** the user types (words, phrases, full sentences), not just entries that happen to already exist in your ~thousands-word local dictionary.

Fix — add an AI fallback path to `/cari`:

- When `kamus.cariKataGUI()` (or `cariKataReverse()` for Indonesian→English) returns no match, **don't just return empty** — call `AIService.askAI()` with a translation-specific prompt instead, e.g.:
  > "Translate this to [English/Indonesian] and reply with ONLY the translation, no explanation: {input}"
- Return the AI result to the frontend the same way a normal dictionary hit would, but tag it (e.g. `"source":"ai"` vs `"source":"local"`) so `dictionary.js` can show a small label like *"Diterjemahkan oleh AI"* (translated by AI) — this is honest UX and also useful for your report (shows you're being transparent about what's a verified local entry vs. an AI-generated one).
- Consider supporting **multi-word input** specifically — right now `Kamus`'s prefix maps are built for single words, so make sure the `/cari` handler doesn't reject or mangle input that contains spaces before it reaches the AI fallback.
- Keep the **existing local dictionary as the first, fast, free lookup** — only fall back to AI (which costs API credits and is slower) when there's no local match. This also ties in nicely with item #4 below (locally-missing words become dictionary contribution candidates).

Files affected: `SimpleAPI.java` (`/cari` handler), `AIService.java` (maybe add a dedicated `translate()` method separate from the generic `askAI()`, with its own prompt template), `js/dictionary.js` (handle the `source` field, show the AI-translated label).

3. Fix: "Network error when trying to fetch resource" on Avatar/Profile Update

Found the actual root cause — this isn't a backend problem at all. In `js/profile.js`, the constant `PROFILE_API` is declared **twice** with `const`:

```
js

// Line 5
const PROFILE_API = (window.location.protocol === 'file:' ...) ? 'http://localh

// Line 343
const PROFILE_API = (window.location.protocol === 'file:' ...) ? 'http://localh
```

In JavaScript, redeclaring a `const` in the same scope throws `SyntaxError: Identifier 'PROFILE_API' has already been declared` — which **breaks the entire script from running**, not just the second declaration. That's why the save/avatar-update button produces a generic "network error": the fetch call never gets a properly initialized `PROFILE_API` to hit, or the click handler never even attaches correctly, because the whole file failed to parse.

Fix: delete the second `const PROFILE_API = ...` block (around the "RESET ALL" section) — keep only the one at the top of the file. Both blocks are byte-for-byte identical, so nothing else needs to change.

Second thing worth checking while you're in there: the non-localhost fallback URL is hardcoded to a specific free ngrok tunnel address (`https://alright-washday-amicably.ngrok-free.dev`). Free ngrok URLs are **temporary and change** every time the tunnel restarts — if that tunnel isn't currently running with that exact address, you'd get a real network error too, separate from the syntax bug above. Worth confirming this URL is still live, or better, load it from one shared config value instead of hardcoding it in every JS file.

Files affected: `js/profile.js` (remove duplicate declaration), verify the ngrok URL is current.

---

4. Bonus Point System — User-Contributed Dictionary Words (AI-Verified)

Goal: let users submit new word pairs (English + Indonesian) to grow the dictionary, get an XP reward immediately for contributing, while an AI check happens in the background to decide whether the word actually gets added to the real dictionary.

Suggested placement: on the Dictionary page — when a search returns no local match (ties in with item #1/#2's AI fallback), show a prompt like *"Kata ini belum ada di kamus kami. Tambahkan sekarang dan dapatkan bonus poin!"* (this word isn't in our dictionary yet — add it now for bonus points!) with a small form (English word, Indonesian meaning, optional example sentence). A quiz-completion placement also works fine if you'd rather keep it separate — either is reasonable, this just felt like the most natural moment for a user to actually have a new word in mind.

Flow:

1. User submits a word pair via a new endpoint, e.g. `POST /dictionary/contribute`.
2. **Server awards XP immediately** (e.g. +10 XP, reuse the existing `updateUserXP()` additive pattern from `/quiz/submit`) and saves the submission to a new `word_contributions` table with status `pending` — then responds to the user right away with success + the XP gained. This is what makes it feel instant.
3. **In the background** (a separate thread, not blocking the response above — e.g. `new Thread(() -> { ... }).start()` or a small `ExecutorService`, since `SimpleAPI` already handles requests concurrently), call `AIService` with a verification prompt like:
   > "Is '{english}' correctly translated as '{indonesian}' in Bahasa Indonesia? Reply with ONLY 'valid' or 'invalid'."
4. If AI says **valid** → insert the word into the real `Kamus` in-memory maps (and persist it — check whether `Kamus` currently loads only from `kamus.db`/`Kamus.txt` at startup, and add a method to also insert new entries at runtime so they're searchable immediately without a server restart), update `word_contributions.status = 'approved'`.
5. If AI says **invalid** → update `word_contributions.status = 'rejected'`, do **not** add it to the dictionary. Per your spec, the user keeps the XP they already got — that's a deliberate UX tradeoff, but worth knowing: it does mean someone could submit junk repeatedly just to farm XP without a single word ever getting approved. Two lightweight, optional guardrails if this becomes a problem later: cap contributions to something like 5/day per user, or make the XP reward smaller than quiz/typing rewards so it's a nice bonus, not the most efficient way to grind XP.
6. **Optional nice-to-have:** show the user's contribution status somewhere on their profile (e.g. "12 kata diajukan, 9 disetujui") — gives a sense of progress and makes the AI-verification step visible/trustworthy rather than invisible.

Files affected:

- `DatabaseManager.java` — new `word_contributions` table (`id`, `user_id`, `english`, `indonesian`, `example_sentence`, `status`, `ai_reasoning`, `submitted_at`).
- `SimpleAPI.java` — new `POST /dictionary/contribute` endpoint (award XP + save pending + spawn background AI check).
- `Kamus.java` — a method to insert a new word into the live in-memory maps at runtime (not just at startup load).
- `AIService.java` — reuse `askAI()` with a new verification-specific prompt, or add a small dedicated `verifyWordPair()` method for clarity.
- `dictionary.html` / `js/dictionary.js` — the "add this word" form, shown when a search comes up empty.
- `profile.html` / `js/profile.js` — optional contribution stats display.

Suggested Order of Work

1. **#3 first** — it's a one-line deletion and fixes a real, currently-broken feature. Do this immediately regardless of anything else.
2. **#1 & #2** — the AI translator fallback for `/cari`. This is what your lecturer specifically asked for, so it's the priority feature item.
3. **#4** — builds naturally on top of #1/#2 (both deal with "word not found locally"), so doing them in this order lets you reuse the same "not found" UI state for both the AI-translate display and the "contribute this word" prompt.